

# Введение в высоконагруженные системы

**Семён Мартынов, Кирилл Дикалин, Георгий Семёнов**

**ИНСТИТУТ ПРИКЛАДНЫХ КОМПЬЮТЕРНЫХ НАУК**

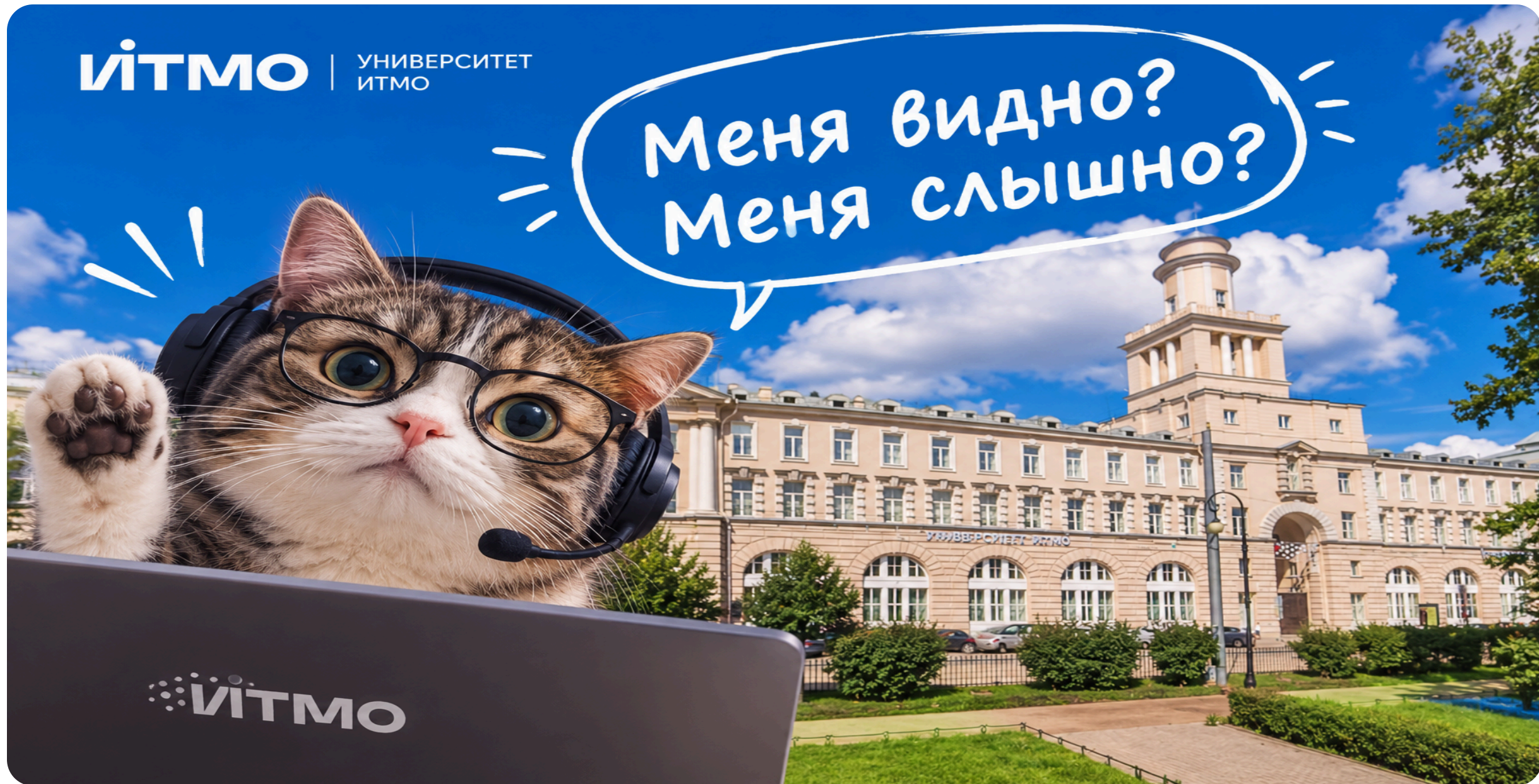
Национальный исследовательский университет ИТМО

**1 октября 2026**

**ИТМО** | УНИВЕРСИТЕТ  
ИТМО

Меня видно?  
Меня слышно?

**ИТМО**



0. Знакомство и требования курса

1. Что такое высоконагруженная система

2. Latency vs throughput

3. RPS/QPS и concurrency

4. Почему среднее — плохая метрика: перцентили и tail latency

5. От метрик к контрактам: SLI → SLO → SLA, error budget



**Семён Мартынов**

**MSE (АУ РАН)**

[in/semen-martynov](#)



**Кирилл Дикалин**

**DevOps (ИТМО)**

[in/kirilldikalina/](#)



**Георгий Семёнов**

**MSE (ИТМО)**

[in/gvsemenov](#)



## Лекции

1. Введение в высоконагруженные системы
2. Очереди и пределы производительности
3. Масштабирование
4. Фундаментальные ограничения распределённых систем для highload
5. Архитектура высоконагруженных систем
6. Хранилища, очереди и брокеры сообщений
7. Устойчивость и управление нагрузкой
8. Системы, ориентированные на пропускную способность
9. Тестирование высоконагруженных систем
10. Системы с низкой задержкой и высокой конкурентностью

## Семинары

1. Мониторинг высоконагруженных систем.
2. Введение в многопоточное программирование.
3. Docker и Kubernetes.
4. Системы управления конфигурациями и координация.
5. REST API, gRPC, балансировка и кэширование.
6. Kafka и RabbitMQ.
7. Паттерны устойчивости.
8. Системы работы с большими данными.
9. Испытания и эксплуатация высоконагруженных систем.
10. Низкоуровневые оптимизации и профилирование.

# Требования и ограничения

- 0. **Обратная связь** (опционально): анонимная гугл-форма после каждой лекции и семинара.
- 1. **Микро-тесты**: 5 минут на каждой лекции (начиная с W2) по материалам прошлой недели.
- 2. **Проекты**: сдача проектных заданий (по направлениям).
- 3. **Экзамен**: итоговая сдача (по направлениям).

| Выполненные условия                           | Макс. оценка                                                                                   |
|-----------------------------------------------|------------------------------------------------------------------------------------------------|
| 3 проектные работы <b>И</b> 9 тестов          | 5                                                                                              |
| Только 2 проектные работы <b>ИЛИ</b> 7 тестов | 4                                                                                              |
| Только 1 проектная работа <b>ИЛИ</b> 5 тестов | 3                                                                                              |
| Прочие случаи                                 | Недопуск  |



Использовать можно, но при ответственном подходе

# 1. Что такое высоконагруженная система

*Курс посвящён проектированию, масштабированию, эксплуатации и исследованию высоконагруженных систем.*

Что такое  
"высоконагруженная  
система?"

# 1. Что такое высоконагруженная система

**Абсолютного порога «высокой нагрузки» не существует.**

Система высоконагружена, когда **наивная реализация перестаёт справляться** и архитектура становится предметом проектирования.

- 100 RPS с тяжёлыми запросами к графовой БД — может быть «высокая» нагрузка
- 100 000 RPS отдачи статики из кэша — может быть рутина

Вопрос не «сколько запросов?», а «какой ресурс упирается первым и что тогда происходит с пользователями?»

# 1. Оси нагрузки

| Ось                   | Что растёт                                  | Пример                          |
|-----------------------|---------------------------------------------|---------------------------------|
| Интенсивность         | запросов в единицу времени                  | распродажа, push-рассылка       |
| Объём работы          | стоимость одного запроса                    | поиск с ранжированием           |
| Объём данных          | размер хранимого и передаваемого            | логи, видео, индексы            |
| Concurrency           | число одновременных запросов/<br>соединений | WebSocket, long polling         |
| Неравномерность       | пик/среднее, burst                          | трансляция матча                |
| Географию и<br>состав | регионы, смесь типов запросов               | чтение/запись, горячие<br>ключи |

Оси **независимы**: рост по одной не означает рост по другой.

# 1. Профиль нагрузки (workload profile)

Система описывается не «числом», а **профилем**:

- **Входной поток:** интенсивность  $\lambda$ , закон прихода (равномерный, пуассоновский, всплески)
- **Смесь запросов:** доли типов, соотношение чтение/запись
- **Объём работы на запрос:** CPU-время, число обращений к хранилищу, размер payload

Без профиля фраза «система выдерживает 5000 RPS» **не имеет смысла**.

# 1. Модель генерации нагрузки

Важнейшая характеристика профиля — как именно источник отправляет запросы:

## Открытая модель (Open model)

- Запросы приходят *независимо* от ответов (по расписанию или внешним событиям).
- Типично для публичных API и веб-трафика.
- Если система не успевает, очередь бесконечно растёт.

## Замкнутая модель (Closed model)

- Клиент *ждет ответа*, прежде чем послать следующий запрос.
- Типично для пулов соединений, батчевой обработки, скриптов.
- Если система замедляется, входящий RPS падает (нагрузка сама себя балансирует).

# 1. Средняя и пиковая нагрузка

- Ёмкость считают по **пику**, а не по среднему
- Пик/среднее: суточные циклы, события, повторные попытки (retry storm)
- Коэффициент неравномерности:  $k = \frac{\lambda_{peak}}{\lambda_{avg}}$

Пример: среднее 1000 RPS,  $k = 5 \rightarrow$  запас нужен минимум под 5000 RPS (и ещё сверху на рост и отказы).

Запас под пик стоит денег  $\rightarrow$  появляются autoscaling, очереди, load shedding (далее по курсу).

# 1. Классы ограничений

| Ресурс            | Как упирается                         | Что смотрим                |
|-------------------|---------------------------------------|----------------------------|
| CPU               | очередь на исполнение                 | utilization, run queue     |
| Память            | своп, OOM, давление GC                | RSS, паузы GC, page faults |
| Ввод-вывод (диск) | очередь к устройству                  | IOPS, latency, queue depth |
| Сеть              | полоса, потери, RTT                   | bandwidth, retransmits     |
| Хранилище         | блокировки, горячие ключи, репликация | lock wait, replication lag |

**USE** (Brendan Gregg): для каждого ресурса — Utilization, Saturation, Errors.

## 2. Latency vs throughput

**Latency** — время обработки **одного** запроса:  $t_{end} - t_{start}$ .

**Throughput** — число запросов, завершённых **в единицу времени**.

- Latency — про опыт одного пользователя
- Throughput — про суммарную работу системы

Это **разные оптимизационные задачи**:

- Снизить latency: кэши, меньше сетевых прыжков, параллелизм внутри запроса
- Поднять throughput: батчинг, конвейеризация, больше воркеров

## 2. Конфликт latency и throughput

**Батчинг:** группа из 100 записей на диск выгоднее, чем по одной (throughput ↑), но первая запись ждёт заполнения батча (latency ↑).

**Загрузка и очередь.** В простейшей модели очереди M/M/1 время в системе

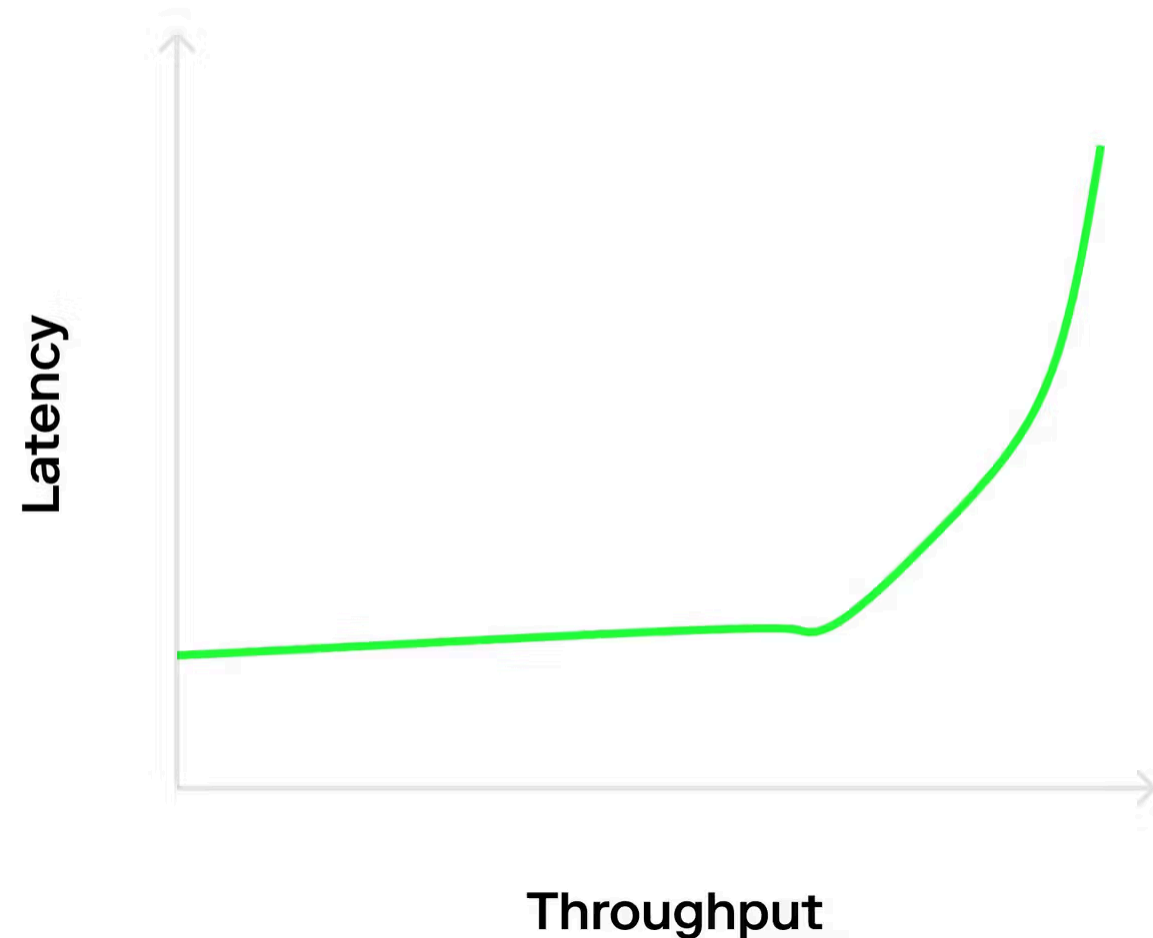
$$W = \frac{1/\mu}{1 - \rho}, \quad \rho = \frac{\lambda}{\mu}$$

| $\rho$        | 0.5 | 0.9 | 0.99 |
|---------------|-----|-----|------|
| $W / (1/\mu)$ | 2×  | 10× | 100× |

Максимум throughput достигается у границы насыщения, где latency **растёт нелинейно** («хоккейная клюшка»). Разберём строго в следующих лекциях.

## 2. Конфликт latency и throughput

**Батчинг:** группа из 100 записей на диск выгоднее, чем по одной (throughput ↑), но первая запись ждёт заполнения батча (latency ↑).



### 3. RPS / QPS и concurrency

- **RPS** — requests per second;
- **QPS** — queries per second (чаще БД и поиск);
- **TPS** — transactions per second

Подвох: единица «запрос» не определена однозначно.

- Один HTTP-запрос может породить десятки обращений к БД
- «Транзакция» в разных системах — разные вещи
- RPS на входе  $\neq$  RPS к хранилищу

Всегда уточняем: **что** считаем, **где** измеряем, **на каком окне** усредняем.

### 3. Concurrency

**Concurrency  $L$**  — сколько запросов находится в обработке **одновременно**.

Анонс, **закон Литтла** (подробно — на следующих лекциях):

$$L = \lambda \cdot W$$

При одном и том же  $\lambda = 1000$  RPS:

| Средняя задержка $W$ | Concurrency $L$ |
|----------------------|-----------------|
| 10 мс                | 10              |
| 50 мс                | 50              |
| 1 с                  | 1000            |

RPS не сводится к concurrency: рост latency при том же RPS **поднимает** число одновременных запросов (потoki, соединения, память).

## 4. Почему среднее — плохая метрика

Выборка из 1000 запросов: 980 по 10 мс и 20 по 1000 мс.

- Среднее:  $\frac{980 \cdot 10 + 20 \cdot 1000}{1000} = 29.8$  мс
- p50 = 10 мс, p99 = 1000 мс

Ни один запрос не выполнялся за ~30 мс. Среднее **не описывает ни один реальный запрос** и скрывает 2% пользователей с задержкой в 100× больше.

Распределения latency — **с тяжёлым правым хвостом** (GC-паузы, холодный кэш, ретраи, конкуренция за ресурсы), а не нормальные.

## 4. Перцентили

**р-перцентиль:** значение, которого не превышают р% запросов.

| Метрика       | Читаем как          |
|---------------|---------------------|
| p50 (медиана) | типичный запрос     |
| p95           | 1 из 20 медленнее   |
| p99           | 1 из 100 медленнее  |
| p99.9         | 1 из 1000 медленнее |

- Перцентили монотонны:  $p50 \leq p95 \leq p99 \leq p99.9$
- Для p99.9 нужно **достаточно данных**: при 1000 запросов в окне это один-единственный запрос
- Смотрим на **max** тоже, но как диагностику, а не как SLO

## 4. Tail latency и опыт пользователя

Страница или API-вызов — это **много** запросов. Вероятность хотя бы одного «медленного»:

$$P = 1 - (1 - q)^N$$

Если 1% запросов к серверу медленные ( $q = 0.01$ ), а запрос разветвляется на  $N = 100$  серверов:

$$P = 1 - 0.99^{100} \approx 63\%$$

Так описывает проблему статья «**The Tail at Scale**» (Dean, Barroso, 2013): при fan-out хвост отдельного сервиса становится **типичным** опытом.

Активные пользователи делают больше запросов → чаще попадают в хвост → хвост определяет восприятие.

## 4. Как правильно измерять

- **Нельзя усреднять перцентили** по инстансам или окнам. Правильно — агрегировать **гистограммы** и считать перцентиль по объединённому распределению
- Границы бакетов гистограммы задают точность: p99 считается с интерполяцией внутри бакета
- **Coordinated omission**: если нагрузочный клиент ждёт ответ перед следующим запросом, он «пропускает» запросы во время простоя и занижает хвост (Gil Tene, *How NOT to Measure Latency*)
- Указываем окно: «p99 за 5 минут»

## 5. SLI → SLO → SLA

|            | Что это                                           | Пример                                |
|------------|---------------------------------------------------|---------------------------------------|
| <b>SLI</b> | Что измеряем (метрика)                            | доля успешных запросов быстрее 300 мс |
| <b>SLO</b> | Цель для SLI                                      | SLI $\geq$ 99.9% за 28 дней           |
| <b>SLA</b> | Формальное обязательство клиенту, с последствиями | 99.5% в месяц, иначе компенсация      |

- SLO обычно **строже** SLA — остаётся запас
- SLI выбираем по тому, что ценит пользователь: доступность, latency, корректность
- Внутренняя цель (SLO)  $\neq$  внешний контракт (SLA)

## 5. Пример: API оплаты

### SLI

- Доступность: доля запросов с кодом не 5xx
- Latency: доля запросов, обработанных быстрее 300 мс (на стороне сервера)

### SLO (окно 28 дней)

- Доступность  $\geq 99.95\%$
- p99 latency < 300 мс, то есть  $\geq 99\%$  запросов быстрее 300 мс

### SLA

- Доступность 99.9% в месяц, иначе возврат части платы

Почему не «средняя latency < 100 мс»? Потому что оно пропускает хвост из предыдущих слайдов.

## 5. Доступность и «девятки»

Допустимый простой за 30 дней (43 200 мин):

| Цель   | Допустимая недоступность |
|--------|--------------------------|
| 99%    | 7.2 часа                 |
| 99.9%  | 43.2 минуты              |
| 99.99% | 4.32 минуты              |

Каждая лишняя «девятка» — **в 10 раз** меньше бюджета и, как правило, заметно дороже.

$$\text{error budget} = 1 - \text{SLO}$$

## 5. Error budget как инструмент решений

**Error budget** — допустимая доля «плохих» событий за окно.

- Бюджет есть → можно выкатывать фичи, эксперименты, рискованные изменения
- Бюджет исчерпан → замораживаем релизы, занимаемся надёжностью
- Скорость сжигания (burn rate) → алерты: «бюджет на месяц закончится за 2 дня»

Превращает спор «надёжность vs скорость разработки» в **измеримое правило**, согласованное заранее.

Наш курс про то, как научиться находить ответы на инженерные вопросы:

## 1. Как правильно измерять систему?

- Мониторить гистограммы и перцентили вместо средних значений.

## 2. Как находить ограничивающий ресурс (bottleneck)?

- Что упирается первым: CPU, диск, сеть или блокировки в БД?
- Использовать метрики RED / USE.

## 3. Как проверять архитектурные изменения?

- Нагрузочное тестирование (с правильным профилем и моделью).
- Канареечные релизы (A/B тесты на реальном трафике).

Что происходит, когда профиль нагрузки превышает возможности системы:

1. Растёт **concurrency** (всё больше запросов застревает в обработке).
2. Резко растёт **latency** (формируются очереди к ресурсам).
3. Пробиваются **таймауты** у клиентов.
4. Клиенты делают автоматические повторные запросы (**retry storm**).
5. Возникает **каскадный сбой**: падает один узел → нагрузка перетекает на оставшиеся → падают все.

*Задача архитектуры — сделать так, чтобы при перегрузке система деградировала изящно (*graceful degradation*), отбрасывая излишки, а не падала целиком.*

На семинаре вы увидите графики Grafana. Ищем:

| Сигнал                | Вопрос                                            |
|-----------------------|---------------------------------------------------|
| Traffic (RPS)         | какой профиль нагрузки, есть ли пики?             |
| Latency p50/p95/p99   | расходятся ли перцентили? когда отрывается хвост? |
| Errors                | растёт ли доля ошибок вместе с нагрузкой?         |
| Saturation, in-flight | какой ресурс упирается, растёт ли concurrency?    |

Это «четыре золотых сигнала» из книги Google SRE. Запрос в Prometheus:

```
histogram_quantile(0.99,  
  sum by (le) (rate(http_request_duration_seconds_bucket[5m])))
```

- Beyer et al. *Site Reliability Engineering* (Google), главы о SLO и мониторинге
- Kleppmann. *Designing Data-Intensive Applications*, гл. 1